

Adobe Photoshop 2030

AI-Powered Image Editing System

Technical Architecture & Implementation Documentation

TEAM67

Version 1.0
December 4, 2025

Contents

1	Executive Overview	3
2	System Architecture	3
2.1	Architectural Philosophy	3
2.2	Component Overview	3
2.3	Client Architecture (Flutter Frontend)	4
2.3.1	Overview	4
2.3.2	Architectural Roles	5
2.3.3	Layered Design	5
2.3.4	Functional Interaction Flow	5
2.3.5	Client Architecture Benefits	5
3	AI Pipeline Architecture	6
3.1	Hybrid Intelligence Strategy	6
3.2	On-Device AI Components	6
3.3	Cloud AI Components	6
3.3.1	Inpainting and Generative Fill	6
3.3.2	Object Movement and Relocation	7
3.3.3	Background Restoration and Erasure	7
3.3.4	Semantic Style Transfer	7
3.4	Model Routing and Orchestration	7
4	Feature Implementation and Workflows	8
4.1	Image Initialization and Segmentation	8
4.2	Inpainting and Generative Fill	8
4.2.1	Operation Flow	8
4.2.2	Model Configuration and Constraints	8
4.2.3	Performance Characteristics	8
4.3	Object Movement and Repositioning	9
4.3.1	Operation Flow	9
4.3.2	Model Configuration and Constraints	9
4.3.3	Movement Constraints and Best Practices	9
4.4	Object Erasure and Background Restoration	9
4.4.1	Operation Flow	9
4.4.2	Model Configuration and Performance	10
4.5	Style Transfer and Artistic Synthesis	10
4.5.1	Two-Stage Pipeline Architecture	10
4.5.2	Operation Flow	10
4.5.3	Model Configuration and Parameters	10
4.5.4	Style Transfer Capabilities and Limitations	11
5	Performance Optimization Strategies	12
5.1	Flux Model Optimization	12
5.2	Asynchronous Processing and UI Responsiveness	12
5.3	Memory Management and Resource Constraints	13
5.4	Model Container Optimization	13

6	Technology Stack and Justifications	13
6.1	Client Platform: Flutter	13
6.2	Backend Framework: FastAPI	14
6.3	AI Runtime: Python and PyTorch	14
6.4	Infrastructure: Docker and Container Orchestration	14
6.5	Demonstration Interface: TypeScript and Web Technologies	15
7	Model Selection and Evaluation	15
7.1	Selection Methodology	15
7.2	Inpainting Model Comparison	15
7.3	Style Transfer Model Selection	15
7.4	Segmentation Model Trade-offs	16
7.5	Models Evaluated but Not Deployed	16
8	Conclusion and Future Directions	16

1 Executive Overview

The Adobe Photoshop 2030 represents a hybrid intelligence architecture for AI-powered image editing that combines on-device inference for real-time interaction with cloud-based generative AI for high-fidelity transformations. The system delivers professional-grade editing capabilities including object segmentation, intelligent inpainting, semantic object movement, background restoration, and style transfer across mobile and desktop environments.

The architecture operates through a cross-platform Flutter client that handles user interaction and edge processing, a FastAPI backend that orchestrates model execution, and a containerized AI runtime integrating specialized models for distinct editing operations. This design enables responsive user experiences while maintaining output quality comparable to desktop editing suites, with tiered capability offerings for free and professional users.

2 System Architecture

2.1 Architectural Philosophy

The platform implements a distributed architecture that separates concerns across three primary domains: client-side interaction and rendering, backend orchestration and state management, and specialized AI model execution. This separation enables independent scaling of computational resources, modular upgrades to AI capabilities, and consistent user experiences across diverse hardware platforms. The architecture prioritizes responsiveness through edge computing for latency-sensitive operations while leveraging cloud infrastructure for computationally intensive generative tasks.

2.2 Component Overview

The system comprises three architectural tiers that cooperate through well-defined interfaces. The presentation tier, implemented in Flutter, manages user interaction, real-time preview generation, and on-device AI inference. The orchestration tier, built on FastAPI, validates requests, manages editing sessions, and routes operations to appropriate AI services. The execution tier consists of containerized model services that perform specialized generative tasks including inpainting, object manipulation, and style synthesis. Each tier operates with distinct performance characteristics and resource requirements, allowing targeted optimization strategies.

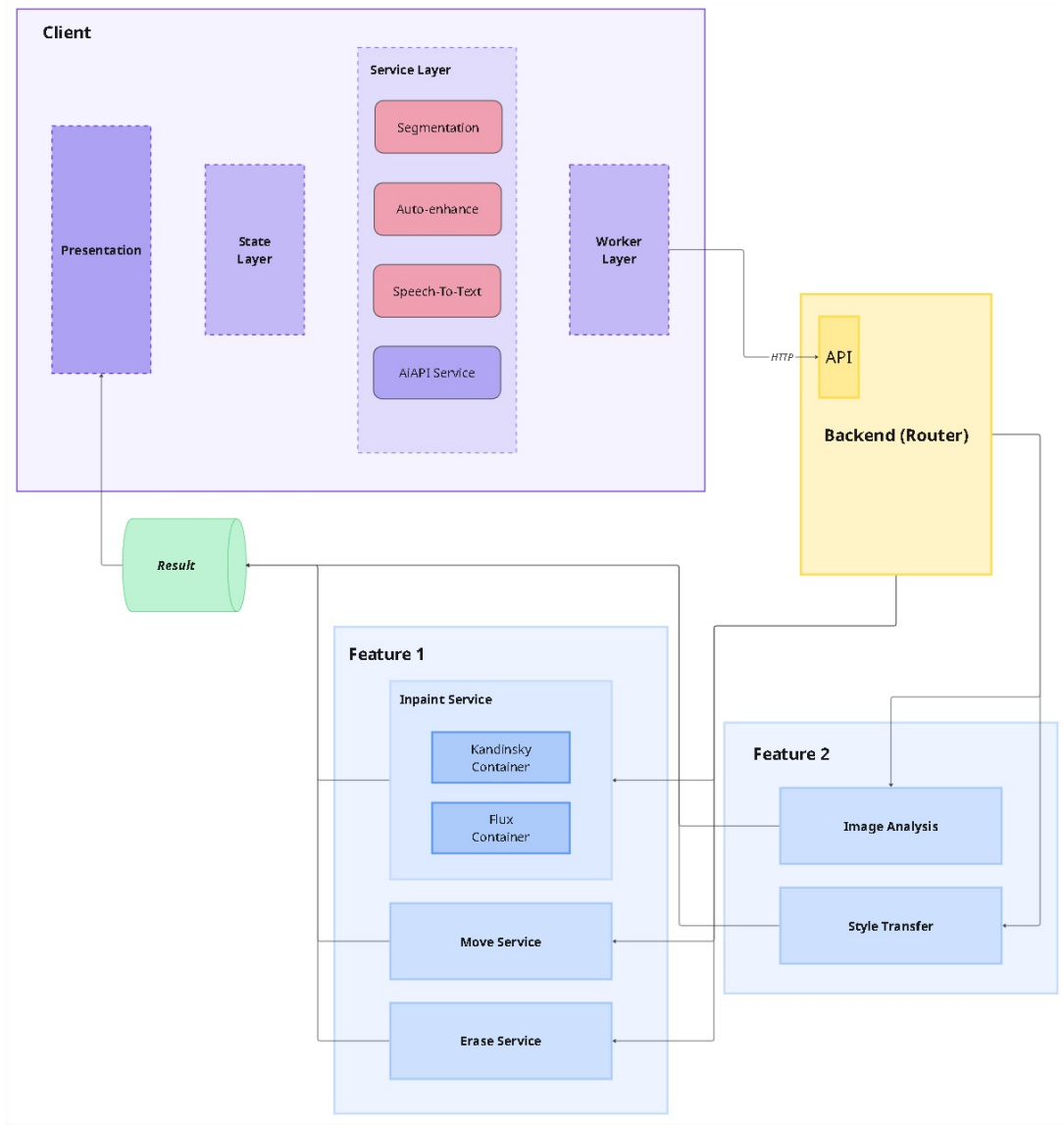


Figure 1: High-level system architecture showing component relationships

2.3 Client Architecture (Flutter Frontend)

2.3.1 Overview

The Flutter-based client serves as the primary interaction layer, implementing a layered architecture that emphasizes responsive rendering and deterministic state management. The presentation layer renders the editing interface with live previews during interaction operations such as object selection and movement. The state management layer employs Riverpod with immutable data models to maintain editing history and enable reliable undo/redo functionality.

The service layer interfaces with on-device AI capabilities for segmentation and communicates with backend services for generative operations. Worker isolates handle CPU-intensive bitmap operations including mask merging and polygon rasterization without blocking the UI thread, ensuring smooth 60 FPS interaction even during complex editing workflows.

2.3.2 Architectural Roles

The client is responsible for rendering the user interface and editing canvas, capturing input actions such as taps, drags, selections, and prompts. It manages editing state and undo/redo history while executing on-device AI inference for fast interaction.

The client also communicates with backend services for heavy generative tasks and handles background image and mask operations without blocking UI threads. This separation of concerns ensures that user interactions remain fluid while complex processing occurs asynchronously.

2.3.3 Layered Design

The client architecture is built around multiple cooperating layers that work together to deliver a responsive editing experience.

Presentation Layer - This layer implements the editor screen, widgets, toolboxes, overlays, and selection interfaces. It provides live previews during interaction such as drag-to-move operations, mask overlays, and brush strokes, giving users immediate visual feedback.

State Management Layer - This layer uses Riverpod and immutable data models including EditState and EditHistory. It records every user transformation as a snapshot for reliable undo/redo functionality. State changes are published through providers to trigger reactive UI updates across the application.

Service Layer - This layer includes on-device segmentation operations using the YOLO model, speech-to-text service for voice commands, auto-enhance service for local processing, and AI API for remote operations. It acts as the bridge between the UI and the underlying AI capabilities.

Worker Layer (Isolates) - This layer offloads CPU-intensive bitmap operations including mask merging, polygon rasterization, and region filling. By running these operations in separate isolates, the UI thread remains smooth and responsive even under heavy computational load.

2.3.4 Functional Interaction Flow

User actions flow through a reactive pipeline where controllers interpret interactions, load required state and mask data, and trigger either local inference or remote processing. When operations can be resolved locally through YOLO inference or isolate-based processing, results propagate through Riverpod state updates that trigger UI rebuilds.

Operations requiring generative AI capabilities invoke backend services, with decoded response images pushed as new state snapshots. This pipeline provides instant responsiveness and state consistency throughout the editing workflow.

2.3.5 Client Architecture Benefits

This architecture delivers cross-platform compatibility across Android, iOS, Windows, and macOS while maintaining low-latency experiences through on-device machine learning. The use of immutable state modeling ensures deterministic behavior, making the application predictable and reliable.

Modular abstraction layers provide ease of scalability and testing, allowing individual components to be developed and tested independently. The combination of isolates

and reactive rebuilding powers a responsive UI that maintains smooth performance even during intensive editing operations.

3 AI Pipeline Architecture

3.1 Hybrid Intelligence Strategy

The AI architecture implements a two-tier execution model that balances latency requirements against output quality constraints. On-device intelligence running directly within the Flutter client via TensorFlow Lite handles operations requiring sub-second response times, primarily object detection and segmentation. Cloud intelligence deployed as containerized services executes computationally expensive generative tasks including inpainting, semantic object movement, and style synthesis. This distribution ensures user interactions remain responsive while maintaining professional-grade output quality for final edits. The separation also allows independent optimization of each tier, with edge models prioritizing inference speed and cloud models focusing on visual fidelity.

3.2 On-Device AI Components

Real-time object selection employs YOLO11n-seg converted to TensorFlow Lite format, executing directly on client devices with typical latency under 200 milliseconds. The model performs instance segmentation to identify discrete objects within uploaded images, generating per-object masks that users can select for editing operations. This on-device execution eliminates network round-trip delays and enables immediate visual feedback as users interact with detected regions. The segmentation pipeline runs asynchronously in background isolates to prevent UI thread blocking, with results propagated through reactive state management once processing completes.

3.3 Cloud AI Components

Cloud-based AI capabilities are deployed as independent containerized services, each optimized for specific editing operations. The architecture supports both free-tier and pro-tier models to accommodate different user segments and hardware availability. Container isolation enables independent scaling, version management, and resource allocation per model type.

3.3.1 Inpainting and Generative Fill

The inpainting pipeline offers tiered quality levels matched to user subscription status and computational budgets. Free-tier users access Kandinsky, a latent diffusion model optimized for standard inpainting and generative fill operations with emphasis on speed and stability. Pro-tier users access Flux running in quantized mixed-precision configuration, delivering higher texture fidelity and visual coherence while fitting within VRAM constraints through architectural optimizations detailed in Section 5.1. Both models accept original images, target region masks, and optional text prompts, generating seamless fills that match surrounding visual context.

3.3.2 Object Movement and Relocation

Semantic object movement employs Dragon Diffusion, a specialized model that preserves object identity and geometric characteristics during spatial relocation. The pipeline accepts the original image, a mask defining the object to move, source coordinates, and target coordinates. The model performs a two-stage operation that first removes the object from its original position while reconstructing plausible background, then reinserts the object at the new location with appropriate perspective, lighting, and occlusion handling. This approach maintains shape consistency and visual coherence throughout the movement operation, typical inference time approximately 21 seconds with 9GB VRAM consumption.

3.3.3 Background Restoration and Erasure

Object removal utilizes ObjectClear, a model optimized for background-aware erasure operations. Unlike general-purpose inpainting models that may introduce unnecessary artifacts or stylistic changes, ObjectClear focuses specifically on coherent background reconstruction after object removal. The model analyzes surrounding context to generate plausible texture and structure continuity, avoiding smearing artifacts common in simpler methods. This specialized approach provides efficient erasure capability for free-tier users without requiring the computational overhead of larger generative models.

3.3.4 Semantic Style Transfer

Style transfer implements a two-stage pipeline combining semantic understanding with high-fidelity rendering. The first stage employs Qwen, a vision-language model that interprets style descriptions and image content to generate detailed text embeddings. Qwen accepts either reference style images or natural language prompts alongside the original image, producing semantic guidance that captures desired artistic attributes. The second stage passes this guidance along with the original image to Flux, which performs style synthesis while preserving content structure. This separation enables more controlled stylistic transformations compared to single-stage approaches, with Qwen-3B selected for its balance of reasoning quality and 2-second inference latency.

3.4 Model Routing and Orchestration

The backend implements routing policies that map editing operations to appropriate model containers based on feature type and user tier. Smart selection operations route to on-device YOLO with optional cloud-based refinement for low-confidence results. Inpainting requests route to Kandinsky for free users and Flux for pro users. Movement operations consistently route to Dragon Diffusion regardless of tier due to its specialized capabilities. Erasure operations utilize ObjectClear exclusively as a lightweight, purpose-built solution. Style transfer operations execute the Qwen-to-Flux pipeline for pro users only. This mapping allows flexible model substitution and A/B testing without client-side modifications.

4 Feature Implementation and Workflows

4.1 Image Initialization and Segmentation

The editing workflow begins with image upload from the client, which sends a set-image request to the backend containing the base64-encoded image data. The backend stores this original image in its workspace and assigns a unique session identifier for subsequent operations. Following successful image storage, the system executes automatic segmentation either on-device using YOLO11n-seg or via scheduled backend processing. The segmentation process generates discrete object masks that represent selectable regions within the image. These masks are transmitted to the backend through a set-masks request, where they are persisted alongside the session reference. The backend returns mask definitions to the frontend, which renders visual overlays enabling user selection of one or multiple objects for subsequent editing operations.

4.2 Inpainting and Generative Fill

4.2.1 Operation Flow

Inpainting operations commence when users select target masks and optionally provide text prompts describing desired fill content. The frontend constructs a request payload containing session identifiers, mask references, and prompt text, transmitting this to the backend inpaint endpoint. The controller validates mask coverage and prompt structure before routing to the appropriate model container based on user tier. For free-tier users, the Kandinsky container receives the masked image region and prompt, executing latent diffusion inference to generate fill content. For pro-tier users, the quantized Flux container performs higher-fidelity generation with enhanced detail preservation. Upon completion, generated content is composited with the original image, encoded, persisted under the session identifier, and returned to the client for display.

4.2.2 Model Configuration and Constraints

Kandinsky operates with default diffusion parameters optimized for speed, typically completing inference within 10-15 seconds. Flux employs an 8-step inference schedule with mixed-precision computation, balancing quality against the 28-second latency requirement. The model runs with 8-bit weight quantization, VAE and transformer modules offloaded to CPU, and text encoder retained on GPU. This configuration reduces VRAM consumption from 35GB to 10GB while maintaining professional output quality. Both models support resolution up to 2048x2048 pixels with automatic downsampling for oversized inputs.

4.2.3 Performance Characteristics

Inpainting quality varies with mask complexity, prompt specificity, and surrounding image context. Kandinsky excels at texture-consistent fills for moderate-complexity regions but may struggle with intricate structural elements. Flux demonstrates superior performance on complex architectural features, human anatomy, and scenes requiring semantic coherence across large masked areas. Inference latency scales approximately linearly with

mask area for both models, with small regions (under 20% of image area) processing near the lower bounds and large regions approaching upper latency limits.

4.3 Object Movement and Repositioning

4.3.1 Operation Flow

Object movement begins with user selection of a source mask and specification of target coordinates through drag-and-drop interaction or explicit coordinate input. The frontend captures starting and ending positions, constructs a movement request containing the session reference, mask identifier, and coordinate pairs, and transmits to the backend move endpoint. The controller validates spatial parameters to ensure the target location falls within image boundaries and the object fits without excessive truncation. Dragon Diffusion receives the original image, source mask, and coordinate transformation, executing a dual operation that removes the object from its original position while reconstructing the background, then reinserts the object at the new location with perspective and lighting adjustments. The backend composites the result, encodes the output, and returns the moved-object image to the client.

4.3.2 Model Configuration and Constraints

Dragon Diffusion operates with a 50-step diffusion schedule optimized for geometric consistency and shape preservation. The model employs attention mechanisms that maintain object identity features including texture, color distribution, and fine details during repositioning. Inference consumes approximately 9GB VRAM and completes within 21 seconds under standard conditions. The model supports movements across the full image canvas but performs optimally when target positions maintain similar scale and viewing angle to source positions. Extreme perspective changes or scale transformations may introduce subtle artifacts that require manual refinement.

4.3.3 Movement Constraints and Best Practices

Successful object movement depends on several factors including object complexity, background homogeneity, and spatial displacement magnitude. Simple objects with clearly defined edges move more reliably than complex, semi-transparent, or highly textured objects. Backgrounds with regular patterns or gradients enable cleaner source removal compared to heterogeneous or high-frequency backgrounds. Small movements (under 30% of image dimensions) produce more convincing results than large displacements that require extensive perspective adjustment. The system automatically validates proposed movements, warning users when parameters fall outside optimal ranges.

4.4 Object Erasure and Background Restoration

4.4.1 Operation Flow

Erasure operations initiate when users select masks for objects they wish to remove from the scene. The frontend transmits an erase request containing the session identifier and target mask references to the backend. The controller routes the request to the ObjectClear container, which analyzes surrounding context to infer plausible background

content. The model generates fill content that maintains texture consistency, structural continuity, and color harmony with adjacent regions. Unlike general inpainting, ObjectClear optimizes specifically for background reconstruction without introducing foreground elements or stylistic variations. The backend receives the erased image, composites if necessary, encodes the result, and returns to the client.

4.4.2 Model Configuration and Performance

ObjectClear operates with a lightweight architecture optimized for fast inference and memory efficiency. Typical processing time ranges from 5-8 seconds depending on mask complexity and background characteristics. The model excels at removing objects from uniform or gradually varying backgrounds including sky, walls, and ground surfaces. More challenging scenarios involve removing objects from highly textured backgrounds or regions where the object occludes significant structural elements. In such cases, the model may produce visible seams or texture discontinuities that require additional inpainting passes.

4.5 Style Transfer and Artistic Synthesis

4.5.1 Two-Stage Pipeline Architecture

Style transfer implements a sequential pipeline that separates semantic interpretation from visual synthesis. The first stage employs Qwen-3B to process either reference style images or natural language style descriptions alongside the original content image. Qwen generates detailed text embeddings that capture artistic attributes including color palette, brush technique, texture characteristics, and compositional elements. This semantic representation provides explicit guidance for the synthesis stage while maintaining content-structure preservation. The second stage passes these embeddings along with the original image to Flux, which performs diffusion-based style rendering. This two-stage approach enables more predictable and controllable results compared to single-model style transfer methods.

4.5.2 Operation Flow

Style transfer begins when users select either a reference style image or enter a natural language style description. The frontend packages the original image alongside the style specification and transmits to the backend stylize endpoint. The controller first routes to Qwen for semantic interpretation, which returns text embeddings describing the target style. These embeddings are then passed to Flux along with the original image for synthesis. Flux executes diffusion inference guided by the semantic embeddings, generating a styled output that preserves content structure while adopting specified artistic characteristics. The backend encodes and returns the final styled image to the client.

4.5.3 Model Configuration and Parameters

Qwen-3B processes inputs with a context window of 2048 tokens, completing semantic interpretation within approximately 2 seconds. The model employs temperature sampling with $T = 0.7$ to balance creativity with adherence to provided style specifications. Flux receives Qwen embeddings as conditioning input for its diffusion process, executing

8 inference steps with classifier-free guidance strength of 7.5. This configuration balances style fidelity against content preservation, preventing excessive transformation that obscures original subject matter. Total pipeline latency approximates 30 seconds including both Qwen interpretation and Flux synthesis.

4.5.4 Style Transfer Capabilities and Limitations

The pipeline successfully transfers a wide range of artistic styles including impressionist painting, watercolor, oil painting, sketch, and various illustration techniques. Natural language descriptions enable fine-grained control over attributes like color temperature, brush stroke visibility, and detail level. The system preserves content structure and recognizable features while applying stylistic transformations. Limitations arise with extremely abstract styles that conflict with content preservation requirements, or when style specifications are ambiguous or contradictory. Very detailed original images may lose fine details during stylization, particularly when target styles employ broad brush strokes or simplified forms.

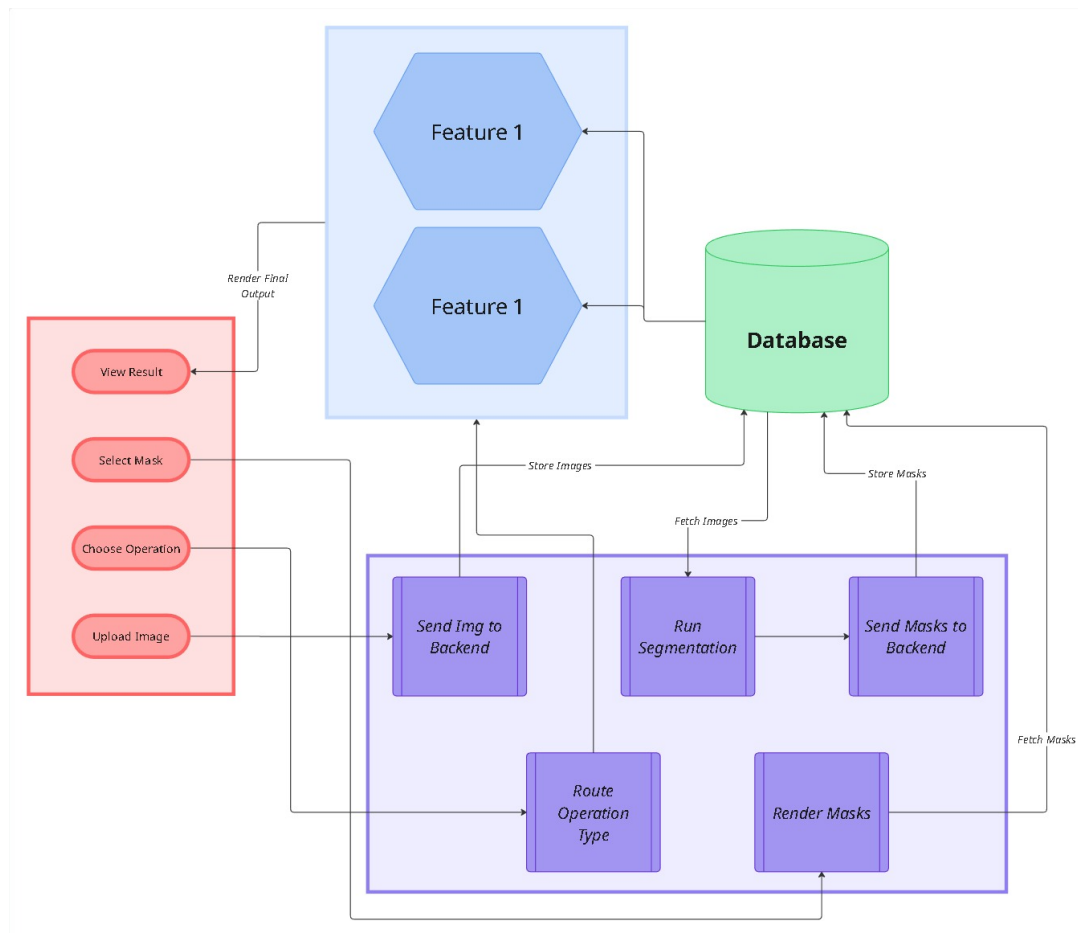


Figure 2: Complete user interaction workflow from image upload to result delivery

5 Performance Optimization Strategies

5.1 Flux Model Optimization

Deployment of Flux for pro-tier inpainting and style synthesis required extensive optimization to meet hardware constraints while preserving output quality. Baseline Flux inference at full 16-bit precision consumed 35GB VRAM and required 75 seconds per operation, rendering deployment infeasible on target hardware. Initial quantization to 8-bit precision using a C++ implementation reduced VRAM to 10GB but increased latency to 840 seconds with unacceptable quality degradation. The final configuration employs selective quantization and strategic module offloading to achieve 28-second latency with 10GB VRAM consumption while maintaining professional output quality.

Table 1: Flux optimization progression and performance metrics

Configuration	Latency (s)	VRAM (GB)	Quality
Baseline 16-bit	75	35	Excellent
C++ 8-bit	840	10	Poor
Mixed Precision (Final)	28	10	Good

The optimized configuration quantizes model weights to 8-bit precision while maintaining critical computations at higher precision. The VAE decoder and transformer blocks are offloaded to CPU, reducing GPU memory pressure while leveraging available system RAM (20-30GB). The text encoder remains on GPU to minimize prompt encoding latency and maintain high-quality text-image alignment. Inference employs 8 diffusion steps, empirically determined as the optimal trade-off between output quality and generation time. This configuration enables **62% latency reduction** and **71% VRAM reduction** relative to baseline while preserving the visual fidelity that distinguishes pro-tier outputs.

5.2 Asynchronous Processing and UI Responsiveness

Client-side performance optimization centers on preventing UI thread blocking during computationally intensive operations. Bitmap operations including mask rasterization, region filling, image decoding, and alpha blending execute in background isolates that run on separate CPU threads. The isolate worker pattern offloads these operations from the Flutter engine thread, ensuring consistent 60 FPS rendering during editing workflows. Results propagate back to the main thread via message passing, triggering reactive UI updates through Riverpod state providers.

The architecture implements optimistic UI rendering for operations with predictable outcomes. When users initiate object selection or mask manipulation, the UI immediately displays preview states while actual processing occurs asynchronously. This approach maintains interaction responsiveness even when operations take several hundred milliseconds to complete. For network-bound operations like generative AI inference, the client displays progress indicators and partial results when available, keeping users informed during the multi-second latency of cloud-based processing.

5.3 Memory Management and Resource Constraints

Mobile deployment imposes strict memory constraints requiring careful management of image assets and editing history. The system limits undo history depth to 6 snapshots, preventing out-of-memory conditions when editing high-resolution images. Each snapshot stores full bitmap data rather than delta operations to enable instantaneous undo/redo without recomputation. When history depth exceeds the limit, the oldest snapshot is discarded. This approach balances functionality against memory availability on target devices.

Image assets undergo automatic resolution management based on device capabilities. Original images above 2048x2048 pixels are downsampled for on-device processing while preserving the full-resolution source for cloud operations. Display rendering uses resolution-appropriate versions to minimize texture memory consumption. These optimizations enable professional editing workflows on mobile hardware while maintaining output quality for final exports.

5.4 Model Container Optimization

Containerized AI services employ several optimization strategies to maximize throughput and minimize resource consumption. Models load on container initialization rather than per-request, eliminating startup latency for each inference operation. Warm standby containers maintain loaded model weights in memory, reducing first-request latency from minutes to seconds. Container orchestration distributes requests across multiple instances based on current load and hardware availability, enabling elastic scaling during usage peaks.

Session-based architecture reduces redundant data transfer by caching uploaded images and masks on the backend. Subsequent operations within a session reference these cached assets by identifier rather than retransmitting full image data. This optimization significantly reduces network overhead for multi-operation workflows where users iterate on the same image with different editing operations.

6 Technology Stack and Justifications

6.1 Client Platform: Flutter

Flutter was selected as the client platform for its unique combination of cross-platform capability and native performance characteristics. The framework enables development of Android, iOS, Windows, and macOS applications from a single codebase, dramatically reducing development effort and maintenance overhead compared to parallel native implementations. Flutter’s GPU-accelerated rendering via Skia provides smooth, low-latency animations and interactive previews essential for professional editing workflows. The hot reload development workflow accelerates iteration cycles, enabling rapid experimentation with new AI features and UI refinements.

Flutter’s plugin ecosystem provides efficient access to platform capabilities including camera interfaces, file systems, GPU computing, and speech input without requiring native code for each platform. The framework’s reactive architecture aligns naturally with the event-driven nature of image editing applications, where user interactions trigger state changes that propagate through the UI hierarchy. These characteristics make

Flutter particularly well-suited for AI-powered creative applications requiring responsive interaction across diverse hardware platforms.

6.2 Backend Framework: FastAPI

FastAPI was chosen for the backend implementation due to its high-performance asynchronous architecture and strong typing discipline. The framework’s `async/await` support enables efficient handling of long-running AI inference requests without blocking server threads, maximizing throughput on compute-limited hardware. Automatic OpenAPI documentation generation provides clean, interactive API specifications that facilitate frontend integration and testing. Pydantic-based request and response validation enforces type safety across the entire API surface, catching integration errors during development rather than production.

FastAPI’s deployment characteristics align well with containerized environments, integrating seamlessly with Uvicorn/Gunicorn for production serving and Docker for environment management. The framework’s relatively small footprint and minimal dependency tree reduce container image sizes and initialization times. These properties make FastAPI an efficient choice for orchestration services that coordinate between client applications and heavyweight AI model containers.

6.3 AI Runtime: Python and PyTorch

Python anchors the machine learning stack due to its dominance in the AI research community and comprehensive ecosystem of libraries. PyTorch provides the foundation for diffusion model implementation, training, and inference with strong GPU acceleration support. The Diffusers library offers pre-built pipelines for Flux, Kandinsky, and other diffusion architectures, accelerating development while maintaining flexibility for customization. NumPy, PIL, and OpenCV handle image preprocessing, postprocessing, and transformation operations with optimized implementations.

Python’s rapid experimentation characteristics enable quick iteration on model configurations, inference parameters, and optimization strategies. The language’s extensive debugging and profiling tools facilitate performance analysis and bottleneck identification. While Python introduces some overhead compared to compiled languages, the performance-critical components (model inference) execute in optimized C++/CUDA code via PyTorch, making Python’s interpretive overhead negligible for this workload.

6.4 Infrastructure: Docker and Container Orchestration

Docker provides environment isolation and reproducibility essential for deploying complex AI stacks with conflicting dependencies. Each model runs in its own container with precisely specified Python versions, library dependencies, and CUDA toolkit versions. This isolation prevents dependency conflicts and enables independent updates to individual models without affecting other system components. Containers can be versioned, tested, and rolled back independently, reducing deployment risk.

GPU-accelerated containers via NVIDIA Container Toolkit enable transparent CUDA access without host system configuration. Development, staging, and production environments achieve parity through container reuse, eliminating environment-specific bugs. Container orchestration enables horizontal scaling of model services, with load balancers

distributing requests across multiple instances based on current utilization. These characteristics make Docker the ideal deployment platform for the heterogeneous AI pipeline required by the Adobe Photoshop 2030 system.

6.5 Demonstration Interface: TypeScript and Web Technologies

TypeScript was selected for the web-based demonstration and monitoring interface due to its type safety and strong tooling support. The language enables rapid development of interactive dashboards that visualize system metrics including GPU utilization, inference latency, model routing decisions, and pipeline performance. Type-safe REST client generation from OpenAPI specifications reduces integration errors and accelerates frontend development. Modern frameworks like React provide component-based architecture suitable for complex visualization interfaces.

The web-based approach enables zero-friction access for stakeholders, eliminating installation requirements while providing real-time visibility into system behavior. Browser-based rendering handles live data feeds, performance graphs, and image result galleries efficiently. TypeScript’s long-term maintainability characteristics support future expansion of monitoring capabilities as the system scales and additional metrics become relevant.

7 Model Selection and Evaluation

7.1 Selection Methodology

Model selection followed a systematic evaluation process balancing output quality, inference latency, resource consumption, and architectural suitability for specific editing operations. Each candidate model underwent testing on representative image datasets covering various complexities, resolutions, and content types. Evaluation criteria included visual fidelity measured through perceptual metrics, inference time measured across multiple hardware configurations, memory consumption profiles, and qualitative assessment of output characteristics relevant to the intended editing operation.

7.2 Inpainting Model Comparison

Initial inpainting evaluation compared Kandinsky, SDXL Inpaint, and quantized Flux configurations. Kandinsky demonstrated stable, artifact-free fills with consistent texture matching at 10-15 second latency. SDXL Inpaint produced acceptable results for simple regions but exhibited inconsistent behavior on complex structures and high-frequency textures, ultimately disqualified due to reliability concerns. Flux at baseline 16-bit precision delivered superior quality but required infeasible resources. The optimized 8-bit mixed-precision Flux configuration emerged as the pro-tier solution, balancing near-baseline quality with practical resource requirements.

7.3 Style Transfer Model Selection

Style transfer pipeline design evaluated single-stage and two-stage approaches. Direct image-to-image style transfer models demonstrated inconsistent content preservation and

unpredictable stylistic interpretation from natural language descriptions. The Qwen-to-Flux pipeline separated semantic interpretation from visual synthesis, improving controllability and result predictability. Initial testing with Qwen-7B showed strong semantic understanding but 45-second latency exceeded acceptable thresholds. Qwen-3B reduced interpretation time to 2 seconds while maintaining sufficient reasoning quality for style guidance, establishing the final configuration.

7.4 Segmentation Model Trade-offs

On-device segmentation compared YOLO11n-seg against RMBG family models. YOLO demonstrated superior instance segmentation with clearer object boundaries and better handling of overlapping objects. RMBG models provided acceptable matting for foreground extraction but insufficient segmentation granularity for selective editing workflows. YOLO’s 1-second inference time and broad object category support made it the definitive choice for the selection pipeline despite slightly larger model size.

7.5 Models Evaluated but Not Deployed

Several models underwent evaluation but were not included in the final system. SuperGlue for point-of-view transformation showed promise but insufficient maturity for production deployment. Various upscaling models demonstrated quality improvements but introduced latency unsuitable for interactive editing. Alternative diffusion architectures including Stable Cascade and SDXL Turbo failed to match Flux quality-latency trade-offs for the pro tier or Kandinsky stability for the free tier. These evaluations inform future development directions as model ecosystems evolve.

Table 2: Final feature-to-model assignments by user tier

Feature	Free Tier	Pro Tier
Segmentation	YOLO (On-Device)	—
Inpainting	Kandinsky	Flux (Quantized)
Object Movement	—	Dragon Diffusion
Object Erasure	ObjectClear	—
Style Transfer	—	Qwen + Flux

8 Conclusion and Future Directions

The Adobe Photoshop 2030 demonstrates successful integration of hybrid intelligence architecture for professional image editing workflows. The system balances responsiveness requirements through on-device segmentation with quality expectations through cloud-based generative models. Quantization and architectural optimization strategies enable deployment of state-of-the-art diffusion models on consumer-grade hardware while maintaining professional output quality. The modular, containerized architecture supports independent scaling and upgrading of AI capabilities as the model ecosystem evolves.

Future development directions include integration of additional editing operations such as advanced color grading, perspective transformation, and multi-object composition. Research into more efficient diffusion architectures may enable higher-quality

outputs within existing latency budgets or equivalent quality at reduced computational cost. Expansion of on-device capabilities through quantized model deployment could reduce reliance on cloud processing for common operations. The architectural foundation established by this MVP supports these enhancements while maintaining the responsive, professional editing experience users expect from modern creative tools.

References

- [1] Mykhaylo Andriluka et al. *2D Human Pose Estimation: New Benchmark and State of the Art Analysis*. June 2014.
- [2] AI-Forever. *Kandinsky: Text-to-Image Diffusion Model*. GitHub Repository. 2023.
- [3] Black Forest Labs. *Flux: Fast Unified Latent Cross-attention for High-Fidelity Image Generation*. arXiv preprint. 2024.
- [4] Chong Mou et al. *DiffEditor: Boosting Accuracy and Flexibility on Diffusion-based Image Editing*. 2023.
- [5] Chong Mou et al. *Dragondiffusion: Enabling drag-style manipulation on diffusion models*. 2023.
- [6] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: [1912.01703](https://arxiv.org/abs/1912.01703) [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [7] Pablo Pernias et al. *Wuerstchen: An Efficient Architecture for Large-Scale Text-to-Image Diffusion Models*. 2023. arXiv: [2306.00637](https://arxiv.org/abs/2306.00637) [cs.CV]. URL: <https://arxiv.org/abs/2306.00637>.
- [8] Patrick von Platen et al. *Diffusers: State-of-the-art diffusion models*. <https://github.com/huggingface/diffusers>. 2022.
- [9] Dustin Podell et al. *SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis*. 2023. arXiv: [2307.01952](https://arxiv.org/abs/2307.01952) [cs.CV]. URL: <https://arxiv.org/abs/2307.01952>.
- [10] Qwen Team. *Qwen2.5: A Party of Foundation Models*. Sept. 2024. URL: <https://qwenlm.github.io/blog/qwen2.5/>.
- [11] Ultralytics. *YOLO11 Architecture and TensorFlow Lite Deployment*. Ultralytics Documentation. 2024.
- [12] An Yang et al. “Qwen2 Technical Report”. In: *arXiv preprint arXiv:2407.10671* (2024).
- [13] Jixin Zhao et al. “ObjectClear: Complete Object Removal via Object-Effect Attention”. In: *arXiv preprint arXiv:2505.22636*. 2025.